Listing 6.9. A thread-safe queue with locking and waiting: wait_and_pop()

Listing 6.10. A thread-safe queue with locking and waiting: try_pop() and empty()

Listing 6.11. A thread-safe lookup table

Listing 6.12. Obtaining contents of a threadsafe_lookup_table as a std::map<>

Listing 6.13. A thread-safe list with iteration support

Chapter 7. Designing lock-free concurrent data structures

Listing 7.1. Implementation of a spin-lock mutex using std::atomic_flag

Listing 7.2. Implementing push() without locks

Listing 7.3. A lock-free stack that leaks nodes

Listing 7.4. Reclaiming nodes when no threads are in pop()

Listing 7.5. The reference-counted reclamation machinery

Listing 7.6. An implementation of pop() using hazard pointers

Listing 7.7. A simple implementation of get_hazard_pointer_for_current_thread()

Listing 7.8. A simple implementation of the reclaim functions

Listing 7.9. A lock-free stack using a lock-free std::shared_ptr<> implementation

Listing 7.10. Pushing a node on a lock-free stack using split reference counts

Listing 7.11. Popping a node from a lock-free stack using split reference counts

Listing 7.12. A lock-free stack with reference counting and relaxed atomic operations

Listing 7.13. A single-producer, single-consumer lock-free queue

Listing 7.14. A (broken) first attempt at revising push()